Lab Sheet 1: DDL Commands

CREATE, ALTER and DROP — SQL Answers with Expected Outputs

SQL dialect used: MySQL 8.x

The outputs below are representative MySQL results after running the commands in order. Because the sheet renames Student to Student_Details, later column operations are shown against Student_Details. Task 12 adds Email to Employee so that the requested UNIQUE constraint can be demonstrated.

1. Create a database named CollegeDB.

```
CREATE DATABASE CollegeDB;
USE CollegeDB;
```

Expected output:

Query OK — Database CollegeDB created and selected.

2. Create a Student table with Student_ID, Name, Age, Course and City.

```
CREATE TABLE Student (
Student_ID INT,
Name VARCHAR(50),
Age INT,
Course VARCHAR(50),
City VARCHAR(50)
);
```

Expected output:

Query OK — Student table created.

Column	Datatype
Student_ID	INT
Name	VARCHAR(50)
Age	INT
Course	VARCHAR(50)
City	VARCHAR(50)

3. Create an Employee table with Emp_ID, Emp_Name, Department and Salary.

```
CREATE TABLE Employee (
Emp_ID INT,
Emp_Name VARCHAR(50),
Department VARCHAR(50),
Salary DECIMAL(10,2)
);
```

Expected output:

Query OK — Employee table created.

Column	Datatype
Emp_ID	INT
Emp_Name	VARCHAR(50)
Department	VARCHAR(50)
Salary	DECIMAL(10,2)

4. Create a Department table with Dept_ID, Dept_Name and Location.

```
CREATE TABLE Department (
Dept_ID INT,
Dept_Name VARCHAR(50),
```

```
Location VARCHAR(50)
);
```

Expected output:

Query OK — Department table created.

Column	Datatype
Dept_ID	INT
Dept_Name	VARCHAR(50)
Location	VARCHAR(50)

5. Create a Product table with Product_ID, Product_Name, Price and Quantity.

```
CREATE TABLE Product (
Product_ID INT,
Product_Name VARCHAR(100),
Price DECIMAL(10,2),
Quantity INT
);
```

Expected output:

Query OK — Product table created.

Column	Datatype
Product_ID	INT
Product_Name	VARCHAR(100)
Price	DECIMAL(10,2)
Quantity	INT

6. Create a Customer table with Customer_ID, Customer_Name, Email and Phone.

```
CREATE TABLE Customer (
Customer_ID INT,
Customer_Name VARCHAR(100),
Email VARCHAR(100),
Phone VARCHAR(15)
);
```

Expected output:

Query OK — Customer table created.

Column	Datatype
Customer_ID	INT
Customer_Name	VARCHAR(100)
Email	VARCHAR(100)
Phone	VARCHAR(15)

7. Create a Course table with Course_ID, Course_Name, Duration and Fees.

```
CREATE TABLE Course (
Course_ID INT,
Course_Name VARCHAR(100),
Duration VARCHAR(30),
Fees DECIMAL(10,2)
);
```

Expected output:

Query OK — Course table created.

Column	Datatype
Course_ID	INT
Course_Name	VARCHAR(100)
Duration	VARCHAR(30)
Fees	DECIMAL(10,2)

8. Create a Faculty table with Faculty_ID, Faculty_Name, Subject and Salary.

```
CREATE TABLE Faculty (
Faculty_ID INT,
Faculty_Name VARCHAR(100),
Subject VARCHAR(100),
Salary DECIMAL(10,2)
);
```

Expected output:

Query OK — Faculty table created.

Column	Datatype
Faculty_ID	INT
Faculty_Name	VARCHAR(100)
Subject	VARCHAR(100)
Salary	DECIMAL(10,2)

9. Create a Library table with Book_ID, Book_Name, Author and Price.

```
CREATE TABLE Library (
Book_ID INT,
Book_Name VARCHAR(100),
Author VARCHAR(100),
Price DECIMAL(10,2)
);
```

Expected output:

Query OK — Library table created.

Column	Datatype
Book_ID	INT
Book_Name	VARCHAR(100)
Author	VARCHAR(100)
Price	DECIMAL(10,2)

10. Create a Department table with Dept_ID as the Primary Key.

```
-- Department already exists from Task 4.
ALTER TABLE Department
ADD PRIMARY KEY (Dept_ID);
```

Expected output:

Query OK — Primary key added to Department.Dept_ID.

Constraint	Column
Column	Dept_ID

11. Create a Student table with Student_ID as Primary Key and Name as NOT NULL.

```
-- Student already exists from Task 2.
ALTER TABLE Student
MODIFY Student_ID INT PRIMARY KEY,
MODIFY Name VARCHAR(50) NOT NULL;
```

Expected output:

Query OK — Student_ID is PRIMARY KEY and Name is NOT NULL.

Column	Constraint
Student_ID	PRIMARY KEY
Name	NOT NULL

12. Create an Employee table with a UNIQUE constraint on Email.

```
ALTER TABLE Employee
ADD Email VARCHAR(100) UNIQUE;
```

Expected output:

Query OK — Email column added with a UNIQUE constraint.

Column	Constraint
Email	UNIQUE

13. Create a Product table with a CHECK constraint that Price > 0.

```
ALTER TABLE Product
ADD CONSTRAINT chk_product_price
CHECK (Price > 0);
```

Expected output:

Query OK — CHECK constraint added. Price must be greater than 0.

Constraint	Condition
chk_product_price	Price > 0

14. Add a new column Email to the Student table using ALTER TABLE.

```
ALTER TABLE Student
ADD Email VARCHAR(100);
```

Expected output:

Query OK — Email column added to Student.

New Column	Datatype
Email	VARCHAR(100)

15. Add a Phone column to the Student table.

```
ALTER TABLE Student
ADD Phone VARCHAR(15);
```

Expected output:

Query OK — Phone column added to Student.

New Column	Datatype
Phone	VARCHAR(15)

16. Modify the datatype/size of the Name column in the Student table.

```
ALTER TABLE Student
MODIFY Name VARCHAR(100) NOT NULL;
```

Expected output:

Query OK — Name changed to VARCHAR(100) and remains NOT NULL.

Column	New Definition
Name	VARCHAR(100) NOT NULL

17. Rename the Student table to Student_Details.

```
RENAME TABLE Student TO Student_Details;
```

Expected output:

Query OK — Student renamed to Student_Details.

Old Table	New Table
Student	Student_Details

18. Rename the column City to Address.

```
ALTER TABLE Student_Details
RENAME COLUMN City TO Address;
```

Expected output:

Query OK — City renamed to Address.

Old Column	New Column
City	Address

19. Add a Salary column to the Employee table.

```
ALTER TABLE Employee
ADD Salary_2 DECIMAL(10,2);
```

Expected output:

Query OK — Salary_2 added. (Salary already exists from Task 3, so a second name is used to avoid a duplicate-column error.)

Column	Datatype
Salary_2	DECIMAL(10,2)

20. Drop the Phone column from the Student table.

```
ALTER TABLE Student_Details
DROP COLUMN Phone;
```

Expected output:

Query OK — Phone column dropped from Student_Details.

Dropped Column	Phone

21. Drop the Email column from the Student table.

```
ALTER TABLE Student_Details
DROP COLUMN Email;
```

Expected output:

Query OK — Email column dropped from Student_Details.

Dropped Column	Email

22. Remove all records from a table using TRUNCATE.

```
TRUNCATE TABLE Student_Details;
```

Expected output:

Query OK — 0 rows remain in Student_Details.

Table	Rows After TRUNCATE
Student_Details	0

23. Drop the Course table from the database.

```
DROP TABLE Course;
```

Expected output:

Query OK — Course table dropped.

Dropped Table	Course

24. Drop the Customer table from the database.

```
DROP TABLE Customer;
```

Expected output:

Query OK — Customer table dropped.

Dropped Table	Customer

25. Complete Student Table with Constraints

This final example creates a fresh table named Student_Final using all four requested constraints: PRIMARY KEY, NOT NULL, UNIQUE and CHECK. It also demonstrates ALTER and DROP operations.

```
CREATE TABLE Student_Final (
Student_ID INT PRIMARY KEY,
Name VARCHAR(100) NOT NULL,
Email VARCHAR(100) UNIQUE,
Age INT CHECK (Age >= 16),
Course VARCHAR(50),
Address VARCHAR(100)
);

-- ALTER operation
ALTER TABLE Student_Final
ADD Phone VARCHAR(15);

-- DROP operation
ALTER TABLE Student_Final
DROP COLUMN Phone;
```

Expected output:

Query OK — Student_Final created. Student_ID is PRIMARY KEY, Name is NOT NULL, Email is UNIQUE, and Age must be at least 16. Phone is then added and subsequently dropped.

Column	Datatype	Constraint
Student_ID	INT	PRIMARY KEY
Name	VARCHAR(100)	NOT NULL
Email	VARCHAR(100)	UNIQUE
Age	INT	CHECK (Age >= 16)
Course	VARCHAR(50)	—
Address	VARCHAR(100)	—

Final verification commands

```
SHOW TABLES;

DESCRIBE Student_Final;
```

Representative SHOW TABLES output after Tasks 23–24:

Tables_in_CollegeDB
Department
Employee
Faculty
Library
Product
Student_Details
Student_Final

Note: The exact formatting of MySQL's console output can vary by client (MySQL Workbench, command-line client, etc.). The tables above show the expected schema/result rather than a screenshot of a particular SQL client.